VISVESVARAYA TECHNOLOGICAL UNIVERSITY, BELAGAVI

AI & NLP DRIVEN SMART STUDENT DATABASE MANAGEMENT SYSTEM USING MYSQL

Phase-1 Project Report

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
GOVERNMENT ENGINEERING COLLEGE MOSALEHOSAHALLI, HASSAN

May 2026

1. ABSTRACT

The AI & NLP Driven Smart Student Database Management System is a state-of-the-art enterprise-grade academic ERP system that bridges the gap between natural human language and structured relational databases. By leveraging advanced NLP techniques, fuzzy phonetic matching algorithms, and RAG (Retrieval-Augmented Generation), the system translates text and voice queries directly into secure, highly optimized MySQL commands. It addresses crucial bottlenecks in institutional operations, including complex search ambiguities, academic status tracking, robust schema migrations, and full transactional recovery with a multi-layered undo snapshot mechanism. The application delivers premium institutional branding, highly resilient data parsing, and absolute security against SQL injection attacks, redefining digital student administration in accordance with modern pedagogical guidelines.

2. INTRODUCTION

In modern academic settings, administrative tasks demand speed, precision, and data integrity. Traditional ERP solutions rely on complex menu trees, multiple filter selections, and precise SQL matching, resulting in steep learning curves and operational delays. The 'AI & NLP Driven Smart Student Database Management System' is developed to resolve these limitations. By integrating intuitive search with relational MySQL databases, it allows users—both tech-savvy administrators and non-technical staff—to command the system via plain text and spoken language. This Phase-1 report outlines the complete architectural layout, database structures, AI-powered parsing engines, 7-layer fuzzy matching modules, and robust soft-delete pipelines that make this system exceptionally resilient, user-friendly, and highly responsive.

3. PROBLEM STATEMENT

Academic databases suffer from severe operational friction:

1. Traditional search engines fail to find records when users make typos or phonetic naming errors, causing redundant entry or administrative delays.
2. When multiple students share identical or similar names, typical systems automatically merge, overwrite, or arbitrarily display the first match without letting the user select the appropriate student, creating data contamination.
3. Bulk file uploads of spreadsheets containing varied column layouts require manually rearranging spreadsheets to fit rigid database constraints, leading to massive manual parsing effort.
4. Accidentally deleted or updated records are immediately permanent in traditional relational setups, lacking an instant point-in-time recovery without restoring bulky daily database backups.

4. OBJECTIVES

The core objectives of the system include:

- Develop a multi-layer NLP-to-SQL search pipeline utilizing token-level fuzzy, trigram, and phonetic matching to enable highly lenient live typing suggestions (≥0.20 score) and exact submitted queries (≥0.70 threshold).
- Implement an ambiguity detection workflow that intercepts multi-student matches with similar names, blocking unauthorized merges and prompting the user to select the specific student by USN.
- Build an AI Column Mapping and Wide-to-Long Excel Parser that auto-detects headers, matches custom columns to canonical database columns via exact/partial matching, and unpivots wide-format

semester metrics to normalized rows.

• Formulate a VTU Semester Progression Logic to automatically calculate estimated semesters, academic years, lateral entry variances, and graduated status based on parsed USN structures.

• Incorporate an undo-snapshot engine offering a 5-minute transaction recovery window for deletion, updates, and bulk uploads by leveraging soft-deleted JSON snapshots.

5. EXISTING VS PROPOSED SYSTEM

The current project replaces outdated, rigid, and slow search architectures with a state-of-the-art system. The differences are highlighted below:

• SEARCH LOGIC: The existing system utilizes rigid exact SQL substring matching (e.g., LIKE '%name%'), which fails on typos. The proposed system uses a 7-layer NLP pipeline incorporating soundex, double metaphone, trigram, and Levenshtein metrics to resolve spelling and phonetic discrepancies.

• BULK INGESTION: Traditional systems crash or reject spreadsheets if column names do not exactly match the database. The proposed system utilizes a rule-based AI Column Mapper that normalizes column headers and automatically parses wide-format semester matrices to canonical, normalized structures.

• DELETION SAFETY: In typical architectures, database deletions are immediately permanent, risking complete data loss. The proposed system captures full point-in-time pre-deletion JSON snapshots, allowing instant restore via a unique UUID token within a 5-minute window.

• BRANDED EXPORTS: Output reporting in standard tools returns generic unformatted tables. The proposed system employs a unified branded report generator in PDF, Excel, and CSV formats, including institutional headers and computer-filled signature sections.

6. TECHNOLOGIES USED

• Backend Framework: Python FastAPI (v0.109.2) - High-performance ASGI framework enabling concurrent async API execution.

• Frontend Architecture: React.js (v18) & Vite - Lightning-fast UI component loading and responsive design using TailwindCSS.

• Database Engine: Oracle MySQL (v8.0) - Relational DB utilizing indexing, unique keys, and transactions.

• Text & Voice Parsing: Web Speech API - Browser-native voice dictation converting oral requests directly into strings.

• Data Modeling & Math: Pandas (v2.2.0) - High-speed dataframe manipulation used in file parsing.

• Professional Reporting: ReportLab (v4.1.0) & OpenPyXL (v3.1.2) - Direct generation of branded PDF, Excel, and CSV exports.

• Security & Authentication: Bcrypt (v4.1.2) & Python-Jose (v3.3.0) - Robust password hashing and JWT token handling.

7. SYSTEM ARCHITECTURE & WORKING PIPELINE

The operation follows a clear, logical progression:

1. USER QUERY: Plain text input is received via the React input terminal or Web Speech API.

2. NLP PROCESSING: Stop words are eliminated; the query is analyzed for intent (personal, academic, or full) and search terms are extracted.

3. SQL QUERY GENERATION: RAG (Retrieval-Augmented Generation) feeds schema constraints and few-shot examples to the NVIDIA LLM API.

4. SECURITY VALIDATION: The generated SQL is parsed by the security validator. Unsafe commands (DROP, ALTER, TRUNCATE) or destructive statements lacking a WHERE clause are blocked and logged in the security_logs table.

5. DATABASE FETCH: Valid SELECT queries are fetched from the MySQL connection pool. If 0 records are returned, the system falls back to phonetic/fuzzy matching.

6. DUAL-MODAL FEEDBACK: Ambiguities (multi-student name matches) are displayed via selection cards. Confirmed results are mapped and rendered in interactive tables, charts, or exported in high-quality formats.

8. MODULES DESCRIPTION

• NLP Parsing & Intent Detection: Strips stop words, determines query scope (academic vs. personal), and extracts student names/USNs.

• 7-Layer Fuzzy Search Engine: Provides live, lenient, spelling-resilient suggestions during typing and strict confidence scoring upon submission.

• AI File Upload & Column Mapper: Receives CSV/Excel files, normalizes columns, and unpivots wide semester columns into structured rows.

• Unified Export System: Builds identical Report models to construct branded PDFs, Excel sheets, and CSVs with integrated signatures.

• Undo/Snapshot Engine: Intercepts updates, deletes, and uploads to save pre-state JSON dumps for instant Point-In-Time rollback.

9. 7-LAYER NLP SEARCH ENGINE LOGIC

Spelling-resilient matching is achieved using a robust 7-layer hybrid scoring and search pipeline:

1. Exact String Match: Direct case-sensitive string equality checks (100% confidence).

2. Normalized Exact Match: Lowercases input, strips accents via unicodedata, and trims special characters (97% confidence).

3. Prefix & Token Prefix Match: Checks if candidate names or space-separated name tokens start with the query (e.g. 'rut' -> 'ruthik'). Utilizes a length-based coverage score ratio.

4. Substring & Token Substring Match: Checks if normalized query exists inside candidate name tokens (e.g. 'anth' -> 'manjunath').

5. Phonetic Encoding (Soundex & Double Metaphone): Generates standard 4-char English pronunciation keys (Soundex) and advanced multi-pronunciation hashes (Double Metaphone) via Jellyfish to identify words sounding identical despite spelling differences (e.g., 'Sudheer' -> 'Sudhir').

6. Character Trigram Similarity: Extracts character-level trigrams and computes a similarity coefficient matching typos: 2.0 * len(intersection) / (len(trigrams_A) + len(trigrams_B)).

7. Levenshtein & Jaro-Winkler String Distance: Calculates single-character insertions, deletions, substitutions, and transpositions to grade typographical proximity.

• Live Suggestion Rule: Executes leniently with score threshold >= 0.20 for high-response keyboard inputs.

• Submission Execution Rule: Applies strict score threshold >= 0.70 to ensure secure database write-back commands.

10. SMART SEMESTER PROGRESSION LOGIC

The system parses Visvesvaraya Technological University (VTU) USN structures (e.g. '1GC20CS042') using regex:

- Course Duration & Year: First character digit defines course duration (e.g., 4 years). Characters 4-5 represent the short admission year (e.g. 20 -> 2020).
- Student Type: Roll number digits (characters 8-10) are parsed to detect Lateral Entry (roll number >= 400). Regular students start from 1.
- Progression Calculation: Compares admission year to current year. If the current month is >= July, the semester is calculated as (years_diff * 2 + 1) [Odd Semester]. Otherwise, it is (years_diff * 2) [Even Semester]. Lateral entry students automatically receive a +2 semester adjustment.
- Status Mapping: If the estimated semester exceeds (course_duration * 2), status is set to GRADUATED; otherwise, ACTIVE.

11. FILE INGESTION & AI MAPPING

Spreadsheet upload is managed through a strict, zero-loss pipeline:

1. Header Row Detection: Scans the first 20 rows of Excel/CSV sheets to locate the real header by identifying known keywords like USN, name, or GPA.
2. AI Column Mapper: Maps custom headers to canonical database columns. For example, 'Reg Number' maps to 'usn', and 'Fathers Name' to 'father_name'.
3. Wide-Format Unpivoting: Converts wide sheets (columns: sem_1_sgpa, sem_2_sgpa) into standardized normalized rows (Semester, SGPA).
4. USN Validation & Merge: Checks USN structure using standard regex. If valid, records are merged (upserted); duplicate rows are identified and flagged.

12. UNDO & POINT-IN-TIME RECOVERY

To guarantee complete transactional safety, a 5-minute Point-In-Time recovery window is enforced:

- Deletion/Update Interception: Prior to executing any destructive command, the system runs a select query to fetch all target rows.
- Snapshot Generation: Stores the full student profiles and academic marks as a serialized JSON dump in the `global_undo_snapshots` table, returning a unique UUID token.
- Recovery Execution: Upon post-requesting the token to `/api/undo/restore/{token}`, the system deletes the newly modified data, reads the JSON snapshot, and re-inserts the exact original values.

13. MYSQL DATABASE DESIGN

The database is designed with optimized indexing and strict integrity constraints. Key tables include:

- users: Manages credentials, roles (Admin/Staff), and preferences (theme) with custom ENUMs.
- students: Central registry of personal fields (USN, name, DOB, father name, permanent address) with a UNIQUE USN constraint.
- marks: Tracks academic data (SGPA, CGPA) linked to USN via foreign key cascading, protected by a unique index (uq_usn_sem).
- global_undo_snapshots: Stores the pre-operation JSON data, operation type, actor, and status for point-in-time rollbacks.

• query_history & security_logs: Maintain detailed auditing for natural queries, executed SQL statements, and intercepted injection attempts.

14. UI/UX ARCHITECTURE

The frontend is constructed using modular React component design:

• Dashboard Workspace: Provides a single-page reactive shell hosting a collapsible sidebar and unified search hub.

• Live Suggestion Panel: A highly interactive dropdown showing spelling matches with badges (EXACT, FUZZY, PHONETIC).

• Interactive Data Tables: Generates responsive paginated grids with column sorting and search filtering.

• Activity Feed & Rollback Panel: Allows administrators to review security logs and trigger instant undo rollbacks in real time.

15. RESULTS & IMPLEMENTED FEATURES

The Phase-1 implementation succeeds in delivering all core modules:

✔ Verified 7-layer fuzzy phonetic search engine resolving severe typos.

✔ Operational async text and voice-dictated query generation pipeline.

✔ Zero-loss spreadsheet ingestion engine with automatic unpivoting.

✔ Strict security validation blocking malicious AST injection patterns.

✔ Verified 5-minute Point-In-Time Undo Recovery with JSON snapshots.

✔ Institutional branded PDF, Excel, and CSV export modules.

16. REMAINING ISSUES & FUTURE SCOPE

Current limitations and developmental plans for Phase-2 include:

• Current Limitations: Local MySQL service configuration dependencies (requires manual administrative startup); single-instance file upload cache (susceptible to cache expiration on server restart).

• Future Scope: Integration of vector embedding semantic search using Milvus/ChromaDB to handle complex intent mappings; implementation of Redis-based query caching to bypass LLM generation for identical queries; integration of automated WebSockets for multi-user real-time notification feeds.

17. CONCLUSION

Phase-1 of the AI & NLP Driven Student Database Management System successfully establishes a robust, highly responsive, and secure foundation. By combining advanced natural language translation with professional institutional reporting and transaction rollback capabilities, the system reduces administrative workload, prevents data loss, and delivers an exceptionally smooth, premium user experience. The architectural designs and modules verified in this phase will serve as a solid launchpad for advanced AI integrations in future iterations.